# Chapter 6
# Buffer Management

**The cache that decides whether "read a page" costs nanoseconds or milliseconds, and the two policy bits that tie it to durability**

Advanced Data Management

Based on the Chapter 6 lecture and its hands-on section on `adm_lab`

## How to read this chapter

Every cost in Chapters 2 to 5 was counted in pages: a scan costs $B$ page reads, a sort $2B(1+\text{passes})$, a join a formula in $B_R$ and $B_S$, and the optimizer chose the plan with the fewest. All of those page reads go through one component, the **buffer pool**. If the page is already in memory the read is almost free; if not, it costs a disk access that is a thousand to a hundred thousand times slower. This chapter is about that component: what it stores, how it answers a request, how it chooses what to throw away, why a large scan can wreck it and how real systems defend against that, and how its treatment of modified pages connects to crash recovery. The last two sections show how to see all of it in PostgreSQL and work through the problems that exams like to ask. By the end you should be able to compute an effective access time and explain why a 99% hit ratio can still be disk-bound, trace LRU, FIFO and Clock by hand, show Belady's anomaly on a sequence, quantify sequential flooding and the ring-buffer defence, fill the steal/force logging matrix, and read a `Buffers:` line without confusing "read" with "disk".

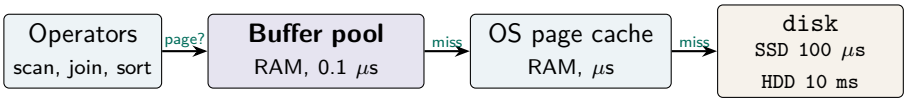
## 1 Why a Cache, and Why the Database Runs Its Own

### 1.1 The memory hierarchy in numbers

Chapter 2 opened with the hierarchy; here are the four numbers this chapter uses everywhere, rounded to the order of magnitude the course's formula sheet quotes.

| Level | Operation | Cost |
|---|---|---|
| RAM | read one 8 KB page already in memory | 0.1 $\mu$s |
| SSD | random read of one page | 100 $\mu$s |
| HDD | random read of one page (seek + rotation) | 10 ms |
| Any disk | sequential streaming | 500 MB/s (64,000 pages/s) |

A memory hit is a thousand times cheaper than an SSD read and a hundred thousand times cheaper than a disk seek. Those ratios, $10^3$ and $10^5$, are the reason the rest of this chapter exists: a system that touches the disk even a small fraction of the time is dominated by that fraction.




## 1.2 Locality and the hit ratio

**Concept: Cache, hit, miss, hit ratio**

A **cache** is a small, fast store holding copies of recently used items from a large, slow store. A request is a **hit** if the item is in the cache and a **miss** if it must be fetched from below. The **hit ratio** $h$ is the fraction of requests that hit. When the cache is full, a **replacement policy** chooses the **victim** to evict.

A cache pays off because of **locality**: *temporal* locality (what was used recently will be used again soon: the root of a B$^+$-tree, the catalog, the current customers) and *spatial* locality (what is near a used item will be used soon: the next page of a scan, the next leaf of a range).

Caching disk pages is the same idea as the OS paging you met in Chapter 2; the differences, developed in §1.4, are who runs the cache and what it knows.

## 1.3 Effective access time: why "99% hit" is not enough

**Mechanism: Effective access time**

If a hit costs $T_{\text{hit}}$ and a miss costs $T_{\text{miss}}$, the average cost of one page access is

$$\text{EAT} \;=\; h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}.$$

Because $T_{\text{miss}} \gg T_{\text{hit}}$, the second term dominates until $1-h$ is tiny: the EAT is governed by the *miss* ratio, not the hit ratio.

**Worked example: The lecture's exercise: how good must the hit ratio be?**

$T_{\text{hit}} = 100$ ns, $T_{\text{miss}} = 10$ ms $= 10^7$ ns (an HDD).
**(1)** $h = 0.99$**.** EAT $= 0.99 \times 100 + 0.01 \times 10^7 = 99 + 100{,}000 \approx 100{,}099$ ns $\approx 100$ $\mu$s. One miss in a hundred makes the average access a thousand times slower than a hit. A 99% cache on a disk is still disk-bound.
**(2) EAT** $\leq 1$ $\mu$**s** $= 1{,}000$ **ns.** Solve $100h + 10^7(1-h) \leq 1000$: $(10^7 - 100)\,h \geq 10^7 - 1000$, so $h \geq 9{,}999{,}000/9{,}999{,}900 \approx 0.99991$. You need **99.991%**: fewer than one miss in ten thousand.
**(3) The 94.9% measured in §8.** $0.949 \times 100 + 0.051 \times 10^7 \approx 510{,}000$ ns $\approx 0.5$ ms per access on an HDD. Fine for a lab demo, hopeless for an OLTP service that promises milliseconds: because a miss costs $10^5$ hits, the few percent of misses *are* the latency.

The same arithmetic on an SSD ($T_{\text{miss}} = 100$ $\mu$s) is kinder but has the same shape. The table shows both; read it column by column and notice that each tenfold cut in the miss ratio buys a tenfold cut in EAT, until the hit cost itself becomes visible.

| $h$ | 0.90 | 0.95 | 0.99 | 0.999 | 0.9999 |
|---|---|---|---|---|---|
| EAT, HDD ($T_{\text{miss}} = 10$ ms) | 1.0 ms | 0.50 ms | 100 $\mu$s | 10.1 $\mu$s | 1.1 $\mu$s |
| EAT, SSD ($T_{\text{miss}} = 100$ $\mu$s) | 10.1 $\mu$s | 5.1 $\mu$s | 1.1 $\mu$s | 0.20 $\mu$s | 0.11 $\mu$s |

**Key insight.** Think in miss ratios. Going from 99% to 99.9% hits is not "one percent better"; it removes nine of every ten disk accesses and divides the average access time by ten.

## 1.4 Why not the OS page cache

The operating system already caches file pages in RAM, and a database is just a process reading files. So why does every serious DBMS keep its own cache, the subject Chapter 2 §6.2 raised and deferred to here? Because the DBMS knows three things the OS cannot:

1. **The access pattern.** The executor knows that a sequential scan will touch each page once and never again, that a nested-loop inner will be re-read on every outer row, that the root of an index is needed by every lookup. A generic LRU cache treats all of these as "recently used". The DBMS can give scans their own small ring (§5), keep hot pages hot, and *prefetch* pages a plan is about to request (turning would-be misses into hits).
2. **The required write order.** Durability (§6) depends on the log reaching disk *before* the data pages it describes. The OS writes back dirty file pages whenever it likes; the DBMS must control when a modified page may leave memory.
3. **Its own memory budget.** Caching the same page once in the OS cache and once in the DBMS is **double buffering**, a waste of RAM; the DBMS sizes its pool deliberately (§7.4) and lets the OS cache cover the rest.

**History.** Michael Stonebraker's 1981 paper "Operating System Support for Database Management" made exactly this argument against the Unix buffer cache; the design of every DBMS since, PostgreSQL included, follows it.

The price is that two caches now sit between a query and the disk (the figure

in §1.1). That is why, in PostgreSQL, a page "read" by the buffer pool may still come from RAM: it missed the pool and hit the OS cache. Keep that in mind for §7.

**Summary**

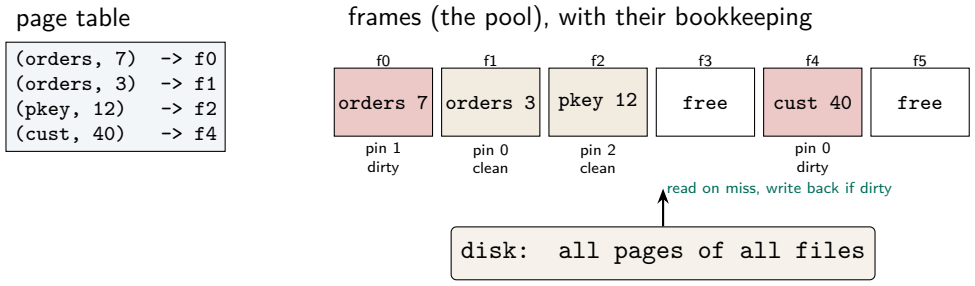
A cache works because of locality and is judged by its hit ratio; but EAT $= hT_{\text{hit}} + (1-h)T_{\text{miss}}$ shows that with a $10^3$ to $10^5$ cost gap the *miss* ratio rules, so OLTP needs 99.9%+ hits. The DBMS runs its own buffer pool rather than trusting the OS cache because it knows the access pattern, must control write ordering for the log, and wants to avoid double buffering.

# 2 The Buffer Pool

## 2.1 Frames and the page table

**Concept: Buffer pool, frame, page table**

The **buffer pool** (buffer cache) is a fixed region of RAM divided into **frames**, each exactly one page in size (8 KB in PostgreSQL). A frame is either free or holds a copy of one disk page. The **page table** (buffer directory) is a hash map from a page identifier (file, block number) to the frame that holds it, if any; it answers the question "is this page resident?" in constant time. The pool caches *every* kind of page: heap pages, index pages, catalog pages, visibility maps.




The pool is *fixed*: it does not grow when a query needs more. PostgreSQL's pool size is the setting `shared_buffers`; its default of 128 MB is $128 \times 1024/8 = 16{,}384$ frames, and a production value of 4 GB is 524,288 frames. The lab's `orders` table, 3,185 pages, fits in the default pool five times over; the review session's 10,000,000-row archive, 63,695 pages, does not fit at all.

## 2.2 Pin count and dirty bit

Each frame carries two pieces of state besides the page itself.

**Concept: Pinned and dirty**

The **pin count** is the number of operators currently using the page. An operator **pins** a page before reading or writing it and **unpins** it when done. A frame with pin count greater than zero *cannot be evicted*: someone holds a pointer into it. A frame with pin count zero is a *candidate* for eviction, nothing more.
The **dirty bit** records that the page in the frame has been modified since it was read, so the copy on disk is stale. A dirty victim must be **written back** (flushed) before its frame is reused; a **clean** victim is simply dropped, because the disk already has it.

Three yes/no questions therefore drive every decision the buffer manager makes about a page: *resident?* (page table), *pinned?* (pin count), *dirty?* (dirty bit). Students mix the last two up with each other and with the usage counter of §4.3; keep them apart. A page can be dirty and unpinned (modified, nobody using it now: evictable, but at the cost of a write). A page can be pinned and clean (being read right now: not evictable, free to drop later). The review quiz's "pin count = 2" means exactly two operators are using the page at this instant; it says nothing about modification or about how often the page was touched.

**In the real world.** In PostgreSQL each frame has a *buffer descriptor* with a tag (which page), flags (valid, dirty), a reference count (the pin count) and a `usage_count` (§4.3). A pin protects the frame from eviction; a separate content lock protects the bytes from concurrent writers. Forgetting to unpin is a classic bug: the frame is leaked until the backend exits.

**Summary**

The pool is a fixed array of page-sized frames plus a page table from page id to frame. Per frame: a pin count (users now; nonzero means not evictable) and a dirty bit (differs from disk; must be written before the frame is reused). Resident, pinned, dirty: three independent facts.

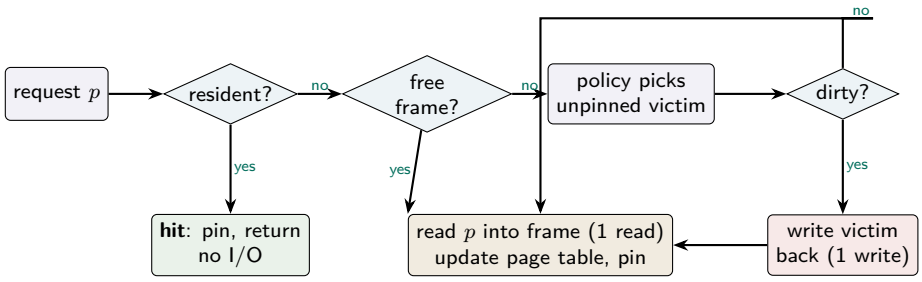
# 3 The Read Path

## 3.1 Requesting a page

**Mechanism: What happens when an operator asks for page $p$**

1. **Look up** $p$ in the page table.
2. **Hit.** $p$ is in frame $f$: increment $f$'s pin count, return a pointer to $f$. No I/O.
3. **Miss.** $p$ is not resident:
   (a) if a **free frame** exists, take it; otherwise ask the **replacement policy** for a victim among the *unpinned* frames;
   (b) if the victim is **dirty**, write it back to disk first (and, in a logging system, make sure the log records for it are on disk, §6.3);
   (c) **read** $p$ from disk into the frame, update the page table (remove the victim's entry, add $p$'s), set pin count to 1, dirty bit to 0;
   (d) return the frame.
4. The operator uses the page and **unpins** it. If it modified the page, it

sets the dirty bit.




## 3.2 What a miss costs

A hit costs no I/O. A miss costs one page read if the victim is clean, and one write plus one read if it is dirty. So a pool full of dirty pages makes every miss twice as expensive; that is why real systems write dirty pages back *in the background* before eviction needs to (§6.1), so that the victim is almost always clean when the policy chooses it. The miss cost also shows why the first pass over any data is unavoidable: a page touched for the first time is a **compulsory miss**; no policy and no amount of memory can turn a first touch into a hit (only prefetching can hide it).

## 3.3 A trace with pins and dirty pages

**Worked example: Three frames, LRU, reads and writes**

Requests, in order: R7, R3, W7, R12, R3, W5, R7 (R = read, W = write; the number is the page). Frames start empty; LRU evicts the unpinned frame whose last use is oldest. Write the pool after each request, oldest on the left, with a star for dirty.

| Request | Result | Pool after (oldest → newest) | Victim and why | I/O |
|---|---|---|---|---|
| R7 | miss, free frame | 7 | | 1 read |
| R3 | miss, free frame | 7, 3 | | 1 read |
| W7 | hit, now dirty | 3, 7* | | 0 |
| R12 | miss, free frame | 3, 7*, 12 | | 1 read |
| R3 | hit | 7*, 12, 3 | | 0 |
| W5 | miss, pool full | 12, 3, 5* | 7: least recently used; dirty, so written first | 1 write + 1 read |
| R7 | miss | 3, 5*, 7 | 12: least recently used; clean, dropped | 1 read |

Seven requests, 2 hits, 5 misses (three of them compulsory), 6 I/Os. Hit ratio $2/7 \approx 29\%$. Two things to notice. First, the write W7 was a hit: writes go to the frame, not to the disk; the disk sees page 7 only when the frame is evicted. Second, if at W5 the operator holding page 7 had

still had it *pinned*, the victim would have been page 12 instead (the oldest *unpinned* frame), and the write-back would have been avoided for now.

**Summary**

Look up; hit means pin and return; miss means find a frame (free, or an unpinned victim chosen by the policy), write the victim back if dirty, read the page, fix the page table. A miss costs one read, two I/Os if the victim is dirty; first touches are compulsory misses. Writes dirty the frame; the disk sees them later.

# 4 Replacement Policies

When the pool is full, which unpinned frame should go? The policy's goal is to maximise future hits; its constraint is that it does not know the future. This section builds from the unattainable optimum to what PostgreSQL actually runs.

## 4.1 The benchmark: Belady's optimal policy

**Concept: OPT**

**OPT** (Belady's optimal, 1966) evicts the page whose *next use is farthest in the future*, or that is never used again. No policy can produce fewer misses on a given request sequence. It needs the whole future, so it is not implementable; it is the yardstick every real policy is measured against, and a useful exam exercise because it is mechanical to trace once the sequence is written down.

The DBMS is in an unusually good position relative to OPT: some of its operators *do* know their own future. A sequential scan knows it will not reuse the page it just read; a nested loop knows it will re-read its inner relation. Chapter 6's defences against flooding (§5) are the buffer manager using exactly that knowledge.

## 4.2 FIFO, LRU and the stack property

**FIFO** evicts the page that was *loaded* earliest, regardless of use. **LRU** (least recently used) evicts the page whose *last access* is oldest: it bets that the recent past predicts the near future, which is what temporal locality says. The distinction matters: under FIFO a page that is hit constantly still ages out when its turn comes; under LRU every hit renews it.

## Concept: Stack algorithms and Belady's anomaly

A policy is a **stack algorithm** if, for every request sequence and every $n$, the set of pages resident with $n$ frames is a subset of the set resident with $n+1$ frames. LRU is one: with $n$ frames it holds exactly the $n$ most recently used pages, which are among the $n+1$ most recently used. OPT is one too. For a stack algorithm, *more memory never causes more misses*.

FIFO is not a stack algorithm, and so it can exhibit **Belady's anomaly**: adding a frame can *increase* the number of misses on some sequences.

**History.** László Bélády described OPT in 1966 and the anomaly in 1969, both at IBM while studying virtual memory; the sequence below is the textbook one from that paper.

## Worked example: The lecture's Exercise 1: Belady's anomaly, every step

Requests: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Frames start empty. In each table the row shows the pool after the request; F is a fault (miss), H a hit.

**FIFO, 3 frames** (oldest loaded on the left):

| request | 1 | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| pool | 1 | 12 | 123 | 234 | 341 | 412 | 125 | 125 | 125 | 253 | 534 | 534 |
| | F | F | F | F | F | F | F | H | H | F | F | H |

**9 faults**, 3 hits (requests 8, 9 and 12).

**FIFO, 4 frames**:

| request | 1 | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| pool | 1 | 12 | 123 | 1234 | 1234 | 1234 | 2345 | 3451 | 4512 | 5123 | 1234 | 2345 |
| | F | F | F | F | H | H | F | F | F | F | F | F |

**10 faults**, 2 hits. One more frame, one more fault: the anomaly. With four frames the early hits on 1 and 2 are followed by a cycle in which every request arrives just after its page was pushed out.

**LRU, 3 frames** (least recently used on the left):

| request | 1 | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| pool | 1 | 12 | 123 | 234 | 341 | 412 | 125 | 251 | 512 | 123 | 234 | 345 |
| | F | F | F | F | F | F | F | H | H | F | F | F |

**10 faults**. **LRU, 4 frames**: 1, 2, 3, 4 fault; 1 and 2 hit; 5 evicts 3 (fault); 1 and 2 hit; 3 evicts 4, 4 evicts 5, 5 evicts 1 (three faults): **8 faults**. More frames, fewer faults, as the stack property guarantees. (With 3 frames LRU is worse than FIFO here, 10 to 9: neither is uniformly better; what LRU guarantees is monotonicity.)

**OPT, 3 frames**, for the yardstick: 1, 2, 3 fault; 4 evicts 3 (next needed at request 10, the farthest); 1, 2 hit; 5 evicts 4 (next at 11); 1, 2 hit; 3 evicts 1 (never needed again); 4 evicts 2 (never); 5 hits. **7 faults**, the minimum possible: 5 compulsory plus 2.

### 4.3 Clock: LRU without the list

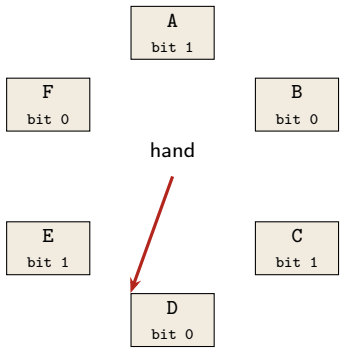
Exact LRU needs an ordered list of all frames, and every hit moves a frame to the front of that list. With many cores touching the pool at once that one list becomes a lock everybody waits on (§4.4). **Clock**, also called **second chance**, keeps LRU's behaviour approximately at a fraction of the cost.

**Mechanism: Clock / second chance**

Frames are arranged in a circle; each has a **reference bit**. A **hand** points at one frame.

- On a *hit*, set the frame's bit to 1. Nothing else moves.
- To *evict*, advance the hand: if the frame under it is pinned, skip it; if its bit is 1, clear the bit to 0 and move on (the frame gets a second chance); if its bit is 0, it is the victim. Load the new page there with bit 1 and leave the hand just past it.

A page that is used at least once per sweep of the hand is never evicted; a page not used during a full sweep is. That is "recently used" with a resolution of one sweep rather than one access.




the hand sweeps clockwise; next eviction: D has bit 0 and is unpinned, so D is the victim; had its bit been 1, the hand would have cleared it and moved on to E

**Worked example: Clock versus LRU on one sequence, and why one bit is not enough**

Three frames, requests A, B, C, A, D, A, B, C. Start with the hand on frame 0 after the three free frames fill (A, B, C, all bits 1).

- A: hit, bit already 1.
- D: miss. Hand on A: bit 1, clear, move. B: clear, move. C: clear, move. Back to A: bit 0, evict A, load D. Pool: D(1), B(0), C(0); hand on B.
- A: miss. B has bit 0: evict B, load A. Pool: D(1), A(1), C(0); hand on C.
- B: miss. C has bit 0: evict C, load B. Pool: D(1), A(1), B(1); hand on D.
- C: miss. Sweep clears D, A, B; back to D with bit 0: evict D. Pool: C(1), A(0), B(0).

Clock: 7 faults, 1 hit. LRU on the same sequence: A, B, C fault; A hit; D evicts B; A hit; B evicts C; C evicts D: 6 faults, 2 hits. Clock did worse

at the request for D: all three bits were 1, the sweep cleared them all, and the victim became A, the page used *most* recently, because a single bit cannot tell "used twice" from "used once". When every bit is set, Clock degenerates into FIFO for that eviction.

PostgreSQL's variant, **clock-sweep**, replaces the bit with a small counter precisely to fix that weakness.

**Mechanism: PostgreSQL clock-sweep: `usage_count` 0 to 5**

Each frame has a `usage_count`. A hit increments it, saturating at 5. A newly loaded page starts at 1. The hand, when looking for a victim, first takes any frame on the free list; otherwise it sweeps: pinned frames are skipped, a frame with count $> 0$ is decremented and passed, a frame with count 0 and pin count 0 is the victim. A page hit many times survives up to five sweeps; a page touched once survives at most one. On the sequence above the counts are A= 2, B= 1, C= 1 when D arrives: the sweep lowers A to 1 and B, C to 0, then evicts B, exactly LRU's choice. The rest of the trace then matches LRU: 6 faults, 2 hits.

## 4.4 Why not exact LRU: the hidden lock

The lesson generalises: hits vastly outnumber misses, so a hit must be nearly free. Clock pays only on misses, when the hand moves.

**In the real world.** A high-core PostgreSQL-like server stopped scaling: throughput flattened as cores were added. The profile showed every backend waiting on the LRU list lock, taken on *every* page hit to move the frame to the front. Clock-sweep touches only the frame's own counter on a hit; after the switch, throughput scaled with cores again.

## 4.5 Beyond recency: frequency and intent

**Trade-offs: Recency is not enough**

**LRU-K** (O'Neil, O'Neil and Weikum, 1993) records the last $K$ access times of each page and evicts by the $K$-th most recent: a page touched once by a scan has no second access time and is evicted before a page touched often, however recently the scan touched it. **2Q** keeps two queues: a page enters a small probationary queue on first touch and is promoted to the main queue only on a second reference, so once-touched pages never displace the hot set. **MRU** (most recently used) is right for one specific case: a loop over a relation larger than the pool (a nested-loop inner). LRU on that loop evicts exactly the page that will be needed next and hits nothing; MRU keeps as much of the relation as fits and hits on all of it. The theme of §5: a good buffer manager uses *frequency* (how often) and *intent* (what the operator is about to do), not only recency.

**Summary**

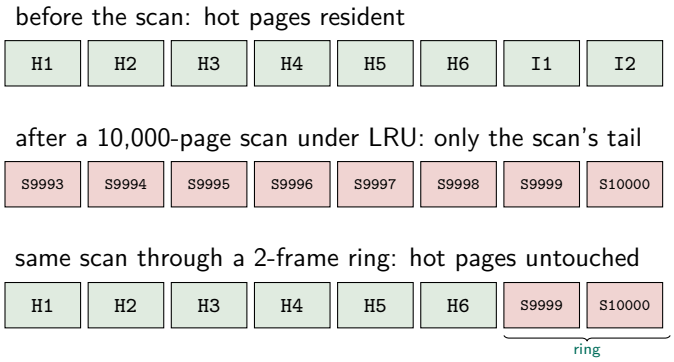
OPT (evict the page needed farthest ahead) is the unattainable yardstick. FIFO evicts the oldest load and can suffer Belady's anomaly; LRU evicts the oldest use, is a stack algorithm (more frames never hurt) but costs

a list update per hit. Clock approximates LRU with a reference bit and a sweeping hand, paying only on misses; PostgreSQL's clock-sweep uses a counter saturating at 5 so that frequency counts. LRU-K and 2Q add frequency; MRU wins for loops larger than the pool.

# 5 Sequential Flooding and Ring Buffers

## 5.1 LRU's blind spot

A large sequential scan reads every page of a table once and never again. Under LRU each of those pages arrives as "most recently used" and is kept in preference to pages that the rest of the workload reuses constantly. Once the scan has read more pages than the pool has frames, every page that was there before has been pushed out: the hot working set is gone, and the next transaction that needs it reloads it from disk, one miss per page. This is **sequential flooding** (scan pollution).




## 5.2 Quantifying the damage

**Worked example: The lecture's Exercise 3**

A pool has $N = 1{,}000$ frames. The workload loops over a hot working set of $W = 200$ pages. A one-pass report then scans $S = 10{,}000{,}000$ pages.
**(1) Hot pages surviving under plain LRU: 0.** The scan streams $10^7$ distinct use-once pages through 1,000 frames. After the first 1,000 scan pages every frame holds a scan page; each hot page's last use is older than every scan page's, so each is evicted in turn. The scan is 10,000 times longer than needed to do this.
**(2) Extra misses on the next loop: 200.** The entire hot set must be reloaded, one compulsory-looking miss per page, although the pages were hot ten minutes ago. At 10 ms per HDD read that is 2 s of pure reloading; across a day of such reports it is the "slow morning".
**(3) With a ring of $r = 32$ frames: all 200 survive.** The scan is

**In the real world.** The "slow morning": a 2 TB nightly report scanned a huge table. Every morning the first hour of transactions was sluggish, because overnight the scan had evicted every hot OLTP page and they all came back from disk one miss at a time. Running the scan through a ring buffer (and off-hours) ended the morning spikes: the hot set never left memory.

confined to its 32 frames and recycles them among itself (page $k$ of the scan evicts page $k - 32$). It never touches the other $N - r = 968$ frames. Since $W = 200 \leq 968$, the hot set is untouched. The cost to the scan is nothing: it would not have hit on its own pages anyway.

**Key insight.** A use-once scan gains nothing from the pool, whatever its size, because every page is a compulsory miss. Giving it a tiny private ring therefore costs the scan nothing and saves everyone else.

## 5.3 Ring buffers in PostgreSQL

**Mechanism: Buffer access strategies**

When the executor starts a **sequential scan of a table larger than `shared_buffers/4`**, it allocates a **ring** of 256 KB (32 frames) and cycles the scan's pages through those frames only. Smaller tables are scanned through the main pool, because a table that fits comfortably is likely to be reused. Bulk writes (`COPY`, `CREATE TABLE AS`) use a 16 MB ring; `VACUUM` uses a 256 KB ring of its own. Index scans and index-driven heap fetches do *not* use rings: their pages are assumed to carry locality.

**Worked example: Is `orders` ringed?**

`shared_buffers` = 128 MB = 16,384 frames; the threshold is 16,384/4 = 4,096 pages (32 MB). The lab's `orders`, 3,185 pages, is below it: a `SELECT count(*) FROM orders` goes through the main pool and leaves all 3,185 pages resident, which is why the second run in §7 shows `hit=3185`. The review session's archive of 63,695 pages is far above it: its scan used a ring and, measured with `pg_buffercache`, occupied only 32 frames while `orders` stayed cached. Lower `shared_buffers` to 64 MB and the threshold becomes 2,048 pages: `orders` is now ringed and a second `count(*)` reads it again.

**Warning / Gotcha: The deck's idealised numbers**

The lecture's hands-on slides assume `orders` has grown to about 5,000 pages and show a cold run with `read=5000` followed by a warm run with `hit=5000`. With the default 128 MB pool a 5,000-page table is *above* the 4,096-page threshold, so a real sequential scan of it would run through a ring and the second run would mostly `read` again. Both pictures are right for their size; the exam question is always "compare the table's pages to `shared_buffers`/4". The lab's actual 3,185-page table behaves as the slides show.

## 5.4 Working sets and diminishing returns

Why does adding RAM help a lot at first and then hardly at all? Because accesses are not uniform. A small **working set** of pages (index roots and upper levels, the catalog, this month's rows, the hottest customers) absorbs most requests; a long tail of pages is touched rarely. The first frames capture the

hot set and the hit ratio climbs steeply; once the pool holds the whole working set, every additional frame caches a cold page that is rarely re-requested, so each added gigabyte buys fewer hits and the curve plateaus. So measure the working set, not the database size (§7.4), and expect one case where more RAM buys nothing: a workload with no reuse, such as one streaming scan of a table far larger than memory, whose hit ratio stays near zero at any pool size.

**In the real world.** The payments API: p99 latency 40 ms with an idle CPU and a busy disk. `pg_statio` showed 86% hits: the hot working set was about 6 GB and `shared_buffers` was 4 GB. Raising the pool until the working set fit took the hit ratio to 99.6% and p99 to 3 ms. The plan never changed; memory, not CPU, was the bottleneck.

**Summary**

A use-once scan under LRU evicts the entire hot set once it exceeds the pool (sequential flooding); the next transactions pay one miss per hot page. A small private ring confines the scan at no cost to it. PostgreSQL rings sequential scans of tables larger than `shared_buffers`/4 (256 KB, 32 frames), bulk writes and vacuum; index scans are not ringed. Hit ratio versus pool size plateaus once the working set fits; a no-reuse workload gains nothing from memory.

# 6 Writing Back: Dirty Pages, Steal, Force and the Log

## 6.1 When dirty pages reach disk

A write modifies a frame and sets its dirty bit; the disk is updated later. Three events write a dirty page back.

**Mechanism: Three ways a dirty page leaves memory**

1. **Eviction.** The policy chose a dirty victim; the backend that needed the frame writes it first (§3.1). This is the expensive case: a foreground query pays a write to do a read.
2. **The background writer.** A process that runs ahead of the clock hand writing dirty frames with low usage counts, so that when the hand reaches them they are clean. In PostgreSQL, `bgwriter`.
3. **Checkpoints.** Periodically (default every 5 minutes, or sooner when `max_wal_size` of log has accumulated) the **checkpointer** writes *all* dirty pages, spread over most of the interval, and records that everything up to this point is on disk. A checkpoint bounds how much log recovery must replay after a crash.

In `EXPLAIN (ANALYZE, BUFFERS)` the first case is the `written` counter; a healthy system shows it rarely, because cases 2 and 3 keep victims clean.

## 6.2 Steal and force

Now bring transactions in. A transaction $T$ dirties pages; it may later commit or abort. Two independent yes/no policies describe how the pool treats $T$'s

dirty pages, and together they decide what the log must contain.

**Concept: Steal / no-steal, force / no-force**

**Steal**: the pool *may* write a dirty page of an *uncommitted* transaction to disk (for instance, to evict it). **No-steal**: it may not; uncommitted changes never reach disk, which means every dirty page of every active transaction must stay pinned in memory until commit.
**Force**: at commit, *all* of $T$'s dirty pages are written to disk before commit is acknowledged. **No-force**: they need not be; commit only has to record the changes somewhere durable and cheap.

Each policy that gives the pool freedom creates a recovery problem. If uncommitted changes can be on disk (steal) and the system crashes before $T$ commits, recovery must **undo** them: it needs the old values. If committed changes can be only in memory (no-force) and the system crashes after $T$ commits, recovery must **redo** them: it needs the new values. The log is where those values live.

**Worked example: The lecture's Exercise 4: the logging matrix**

| | **force** | **no-force** |
|---|---|---|
| **no-steal** | neither undo nor redo | redo only |
| **steal** | undo only | undo and redo |

**Reasoning.** Steal lets uncommitted changes reach disk, so undo information is required. No-force lets committed changes stay off disk, so redo information is required. No-steal + force needs no log at all, and is unusable: no-steal wastes memory (every dirty page of every running transaction pinned until commit; a large update can exhaust the pool), and force is slow (a commit must complete many random writes, one per dirty page, before it can return).
**The cell real systems choose: steal + no-force**, the fastest one: evict whatever the policy likes, and make commit cheap. The price is logging both undo and redo in a **write-ahead log** (WAL), the subject of Chapter 9.

## 6.3 Why steal + no-force needs a write-ahead log

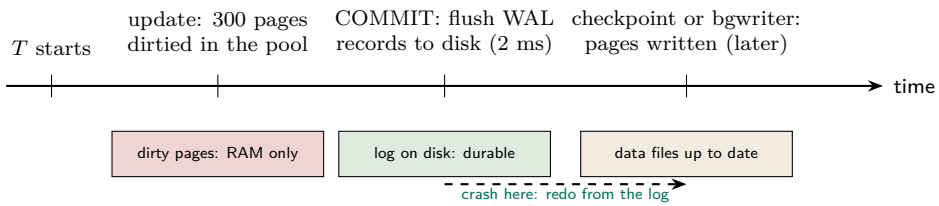
**Mechanism: The write-ahead rule and what commit really writes**

The log is an append-only file of records "transaction $T$ changed page $p$ from old to new". Two rules make steal + no-force safe:

1. **WAL before data.** Before a dirty page is written to disk (stolen or otherwise), all log records describing its changes must already be on disk. Then any change found on disk after a crash can be undone from the log.
2. **Log forced at commit.** At commit, the log up to and including $T$'s

commit record is flushed (`fsync`) to disk. The data pages stay dirty in the pool. Then any committed change missing from the data files after a crash can be redone from the log.

Commit therefore costs *one sequential append* of a few kilobytes, not hundreds of random page writes.




## Worked example: "COMMIT takes 2 ms, but the transaction changed 300 pages. How is that safe?"

At `COMMIT`, PostgreSQL writes and `fsync`s only the WAL records of the transaction, a sequential write of a few KB to the end of the log; the 300 dirty pages ($300 \times 8$ KB $= 2.4$ MB, random positions in the data files) stay in `shared_buffers` and are written later by the background writer or the next checkpoint. That is no-force, and it is what makes 2 ms possible: a forced commit would wait for 300 random writes (3 s on an HDD at 10 ms each; 30 ms on an SSD). After a power cut, recovery starts at the last checkpoint and replays the log: every change of a transaction whose commit record is in the log is **redone** onto the data pages, so the committed transaction loses nothing. A transaction with no commit record in the log is treated as aborted: any of its pages that had been stolen to disk are **undone** (or, in PostgreSQL's MVCC design, its tuple versions are simply never visible), so it leaves no trace.

## Summary

Dirty pages reach disk by eviction (foreground, expensive), the background writer, or checkpoints (all dirty pages, periodically). Steal lets uncommitted pages reach disk and needs undo; no-force lets committed pages stay in memory and needs redo. Real systems run steal + no-force and log both in a write-ahead log: log records before the data page, log forced at commit. Commit is one sequential append; recovery redoes committed work and undoes (or hides) uncommitted work.

**In the real world.** PostgreSQL's WAL carries only redo records. It does not need undo records because MVCC never overwrites a row in place: an update writes a new tuple version and the old one stays on the page. After a crash the tuples of an uncommitted transaction are invisible because its commit status (`pg_xact`) says "aborted", and `VACUUM` reclaims them later. The course's matrix is the general ARIES-style answer; PostgreSQL is the special case where "undo" is free.

# 7 Measuring the Pool in PostgreSQL

## 7.1 The `Buffers:` line

**Concept: Anatomy of a `Buffers:` line**

`Buffers: shared hit=40 read=3145 dirtied=2 written=1, temp read=183 written=183`

- `shared hit`: pages found in `shared_buffers`. No I/O at all.
- `shared read`: pages *not* found in `shared_buffers` and requested from the OS. They may have come from the OS page cache (microseconds) or from the disk (100 $\mu$s to 10 ms); the counter cannot tell.
- `dirtied`: pages this query modified for the first time (writes, or hint bits set by a read).
- `written`: dirty pages this query had to write out itself to free frames. Nonzero means eviction is doing the background writer's job.
- `temp read/written`: pages of temporary files for sorts and hash joins that did not fit in `work_mem`. Nothing to do with the pool; a Chapter 4 symptom (`Batches` $> 1$, external merge).

The hit ratio of one execution is hit/(hit+read). Per node, the line counts that node's pages (cumulative in older versions: read bottom-up); the top node's line is the whole query.

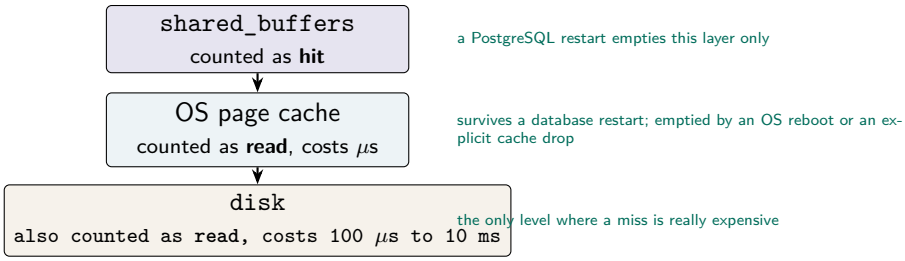
## 7.2 "read" is not "disk"

The mock exam's ticket: the December report ran in 82.9 ms right after a server restart with `shared read=3258`, and in 62.3 ms immediately afterwards with `shared hit=3258`. Three questions.

**Worked example: What is 3,258, why does read become hit, why only 1.3× and not 1,000×?**

**3,258** $= 3{,}185 + 73$: every page of `orders` plus every page of `customers`, read once each by the two sequential scans (the `Hash Join` and the aggregate add nothing; their inputs are already in memory). Both tables are below `shared_buffers`/4, so no ring was used and all 3,258 pages stayed in the pool: the second run finds every one of them, `read` becomes `hit`.

**Why not 1,000× slower cold?** A restart empties `shared_buffers` but not the OS page cache, which still held the files. The 3,258 "reads" were served from RAM by the OS, at microseconds each: 3,258 system calls and copies, about 20 ms, which is the whole difference. Had the pages really come from disk, a sequential read of $3{,}258 \times 8$ KB $\approx 25$ MB would have added about 50 ms at 500 MB/s, and random HDD reads $3{,}258 \times 10$ ms $\approx 33$ s. The 1.3× is the signature of a pool miss that was an OS-cache hit.




**Key insight.** `read` means "not in shared buffers". To know whether it hit the disk, look at the timing (or at `track_io_timing`, which adds `I/O Timings` to the plan), never at the counter.

## 7.3 Three views and one extension

**Interface / API: Seeing the pool**

**Size.** `SHOW shared_buffers;` (default `128MB`). Frames = size / 8 KB.
**Contents**, with the `pg_buffercache` extension, one row per frame with the columns `relfilenode`, `relblocknumber`, `isdirty`, `usagecount` and `pinning_backends`:

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;
SELECT c.relname, count(*) AS buffers,
       pg_size_pretty(count(*) * 8192) AS size,
       sum(CASE WHEN b.isdirty THEN 1 ELSE 0 END) AS dirty
FROM pg_buffercache b JOIN pg_class c ON b.relfilenode = c.
    relfilenode
GROUP BY c.relname ORDER BY buffers DESC LIMIT 5;
```

**Hit ratio per table**, cumulative since the statistics were reset:

```
SELECT relname, heap_blks_hit, heap_blks_read,
       round(heap_blks_hit::numeric / nullif(heap_blks_hit +
    heap_blks_read, 0), 4) AS hit_ratio
FROM pg_statio_user_tables WHERE relname = 'orders';
```

(`idx_blks_hit`/`idx_blks_read` do the same for the table's indexes; `pg_stat_database.blks_hit`/`blks_read` for the whole database.)
**Warming on purpose**: `CREATE EXTENSION pg_prewarm; SELECT pg_prewarm('orders');` reads every page of the table into the pool and returns the count (3,185 for the lab's `orders`). Useful after a restart, before a benchmark, or to make a "warm" measurement reproducible.
**Per query**: `EXPLAIN (ANALYZE, BUFFERS)`, §7.1.

## 7.4 Sizing `shared_buffers`

**Trade-offs: Why the rule of thumb is 25% of RAM, not 80%**

A DBA with 64 GB proposes `shared_buffers = 52GB` because "more cache is always better". Three things go wrong.

- **The OS page cache starves.** PostgreSQL relies on it for everything that is not a pool hit: WAL writes, temporary files, the "read" pages

of §7.2, and the double-buffered copy that makes a pool miss cheap. With 12 GB left for the OS and all other processes, pool misses become disk reads.
- **Sorts and hash joins have no room.** `work_mem` is allocated per operator per connection *outside* `shared_buffers`; 100 connections running a 64 MB sort each need 6.4 GB that no longer exists, so the system swaps or the planner is forced into external sorts and multi-batch hashes (Chapter 4).
- **Checkpoints become storms.** More frames mean more dirty pages accumulating between checkpoints, so each checkpoint writes tens of gigabytes in a burst; latency spikes every few minutes.

Measure before deciding: the per-table hit ratio (`pg_statio`) and the distribution of `usagecount` in `pg_buffercache` (many frames at 4–5: the pool is full of hot pages and could use more; mostly 0–1: it already exceeds the working set). Start at 25% (16 GB here), raise it only while the hit ratio responds (§5.4), and stop near 40%.

**Summary**

`Buffers:` counts pages: `hit` in the pool, `read` not in the pool (OS cache or disk, indistinguishable), `dirtied`, `written` (foreground flushes), `temp` (spills, not the pool). A 1.3× cold/warm gap means the OS cache served the reads. `pg_buffercache` shows the contents, `pg_statio_user_tables` the cumulative hit ratio, `pg_prewarm` loads a table. Size the pool at about 25% of RAM and tune by measured hit ratio and usage counts, not by "more is better".

# 8 Hands-on Companion (`adm_lab`)

The lecture's hands-on section runs four experiments; the numbers below are for the lab's real `orders` (500,000 rows, 3,185 pages) with the default 128 MB pool, with the deck's 5,000-page variant noted where it differs.

**Technique: Part by part**

1. **Pool size and contents.** `SHOW shared_buffers` gives `128MB`, that is 16,384 frames. The `pg_buffercache` aggregate lists only tables touched since the last restart: `orders` with up to 3,185 buffers, `customers` with up to 73. The gap between a table's pages and its buffers is the point: only pages requested so far are resident; the pool fills on demand.
2. **Cold versus warm.** Run `EXPLAIN (ANALYZE, BUFFERS) SELECT count(*) FROM orders WHERE amount > 50` twice after a restart. Run 1: `shared read=3185`; run 2: `shared hit=3185`, noticeably faster. Same plan, same pages; only where they were found changed. If run 1 was only slightly slower, say why (§7.2: the OS cache). With a 5,000-page `orders` and the default pool, expect the ring behaviour

of §5.3 instead: run 2 shows mostly `read` again.

3. **Hit ratio.** The `pg_statio_user_tables` query. The deck's example is 93,000 hits and 5,000 reads: 93,000/98,000 = 0.949. Interpret it with §1.3: fine for a demo, and on an HDD still 0.5 ms per average access; production OLTP wants 99.9%+. Reset with `SELECT pg_stat_reset();` to measure a single experiment.
4. **Warming and the scan that does not flood.** `SELECT pg_prewarm('orders')` returns 3,185. Then scan a table larger than 4,096 pages (create one, or use the review's archive) and re-run the `pg_buffercache` query: the big table occupies only about 32 frames and `orders` is still fully resident. That is the ring buffer of §5.3 observed directly. Scanning `orders` itself (below the threshold) shows the opposite: it is loaded in full and stays.

**Debugging: If the experiment does not show the effect**

- **`pg_buffercache` or `pg_prewarm` does not exist.** They are contrib extensions: install the `postgresql-contrib` package (Linux) or use a distribution that ships them (Homebrew and the official Docker image do). `CREATE EXTENSION` needs superuser or the `pg_read_all_stats` role.
- **Run 1 already shows `hit`.** Something scanned `orders` since the restart (an earlier query, autovacuum, `ANALYZE`). Restart the server again and make the `EXPLAIN` the first statement; or `pg_prewarm` and then measure the warm case only.
- **Run 1 is as fast as run 2.** The OS cache served it; this is normal and is the point of §7.2. To see a true disk read you must also drop the OS cache (Linux: `echo 3 > /proc/sys/vm/drop_caches` as root; macOS: `sudo purge`), which you should do only on your own machine.
- **The big scan evicted everything.** The table was below `shared_buffers`/4 (no ring), or you used an index scan (never ringed). Check `SHOW shared_buffers`, compute the threshold, and make sure the plan says `Seq Scan`.

# 9 Tricks, Techniques and Classic Problems

**Warning / Gotcha: Common student mistakes**

- **"`read` means it came from the disk."** It means it was not in `shared_buffers`; the OS cache may have served it. Use timings to tell.
- **Confusing pin count, usage count and dirty bit.** Pin: users right now, blocks eviction. Usage count: how often touched, decides the victim. Dirty: differs from disk, decides whether eviction costs a write. Three independent fields.
- **Calling 95% a good hit ratio.** Compute the EAT: with a $10^5$ gap,

5% misses means the average access is 5,000 times a hit. Think in miss ratios.

- **Tracing LRU as FIFO.** LRU orders frames by last *use*; a hit moves the page to the newest end. FIFO orders by *load* time; hits change nothing. Belady's anomaly belongs to FIFO, not LRU.
- **"More frames never hurt" for every policy.** True for stack algorithms (LRU, OPT), false for FIFO and for Clock in general.
- **"No-force means committed data can be lost."** No-force means the data *pages* need not be written at commit; the *log* is. Redo restores the pages after a crash. Nothing committed is lost.

**Worked example: Classic exam and interview problems**

**P1. Three frames, requests A B C A B D A B. Hits under LRU? Under FIFO?**
LRU: A, B, C miss; A hit (B C A); B hit (C A B); D evicts C (A B D); A hit (B D A); B hit (D A B): **4 hits**, 4 misses. FIFO: A, B, C miss; A, B hit; D evicts A (B C D); A evicts B (C D A); B evicts C (D A B): **2 hits**. LRU wins because A and B are hot and keep renewing; FIFO ages them out anyway.

**P2. Hit ratio 98%, RAM 0.1 $\mu$s, SSD 100 $\mu$s. EAT? How many times slower than RAM? What hit ratio keeps EAT under 1 $\mu$s?**
$0.98 \times 0.1 + 0.02 \times 100 = 0.098 + 2 = 2.1$ $\mu$s, about 21× a RAM access. For $0.1h + 100(1-h) \leq 1$: $99.9h \geq 99$, $h \geq 0.991$. On an HDD the same target needs 99.991% (§1.3).

**P3. Can adding a frame increase page faults? Give a policy and a sequence; name a policy for which it cannot happen and say why.**
FIFO on 1 2 3 4 1 2 5 1 2 3 4 5: 9 faults with 3 frames, 10 with 4 (Belady's anomaly, §4.2). LRU cannot: it is a stack algorithm, the $n$ most recently used pages are always among the $n+1$ most recently used, so every hit with $n$ frames is also a hit with $n+1$.

**P4. Pool 1,000 frames, hot set 200 pages, one-pass scan of $10^7$ pages. Hot pages left under LRU? Extra misses afterwards? With a 32-frame ring?**
0; 200; 200 survive, because the scan recycles its 32 frames and never touches the other $968 \geq 200$ (§5.2).

**P5. Fill the steal/force matrix and say what real systems choose and why it needs a WAL.**
no-steal+force: neither; no-steal+no-force: redo; steal+force: undo; steal+no-force: undo and redo. Real systems: steal + no-force, the fastest (evict freely, commit with one log append), hence a log with both, written before the data page and forced at commit (§6.2–6.3).

**P6. A plan shows `Buffers: shared hit=40 read=3145`. What does `read=3145` mean, and what is the hit ratio?**
3,145 pages were not in `shared_buffers` and were requested from the OS; some or all may have come from the OS page cache, not the disk. Hit

ratio of this execution: $40/3{,}185 \approx 1.3\%$, a cold scan.

**P7. `shared_buffers` is 128 MB. Is a sequential scan of `orders` (3,185 pages) ringed? Of a 63,695-page archive? What if the pool is 64 MB?**

Threshold = 16,384/4 = 4,096 pages. `orders`: no, it is scanned through the main pool and stays resident. Archive: yes, a 32-frame ring; the rest of the pool is untouched. At 64 MB the threshold is 2,048 pages and `orders` is ringed too: a second `count(*)` reads it again.

**P8. "COMMIT takes 2 ms but changed 300 pages; how is the data safe? What happens after a power cut to a committed and to an uncommitted transaction?"**

Commit flushes only the WAL records (sequential, a few KB); the 300 pages stay dirty in the pool (no-force) and are written by the background writer or checkpoint. After the cut: recovery replays the log from the last checkpoint; a transaction with a commit record has every change redone onto the data pages; one without is treated as aborted and any page of it that reached disk is undone, or, in PostgreSQL, its tuple versions are simply invisible (§6.3).

**P9. 64 GB server, `shared_buffers = 52GB` proposed. Your reply in four lines.**

Claim: no. Mechanism: PostgreSQL also needs the OS page cache (WAL, temp files, misses), `work_mem` outside the pool, and bounded checkpoint I/O. Number: 100 connections at 64 MB of `work_mem` need 6.4 GB; a 52 GB pool can hold tens of GB of dirty pages per checkpoint. Consequence: swapping, external sorts, checkpoint stalls. Start at 16 GB, measure hit ratios and usage counts, raise only while they respond.

## Summary: Chapter 6 in six lines

1. The buffer pool is a fixed array of page-sized frames with a page table; the DBMS runs its own because it knows access patterns, must order writes for the log, and avoids double buffering.
2. EAT $= hT_{\text{hit}} + (1-h)T_{\text{miss}}$: with a $10^3$ to $10^5$ gap the miss ratio rules; 99% hits on an HDD is still 100 $\mu$s per access, and OLTP needs 99.9%+.
3. Per frame: pin count (users now, blocks eviction) and dirty bit (write back before reuse). A miss costs one read, two I/Os if the victim is dirty; first touches are compulsory.
4. OPT is the yardstick; FIFO can show Belady's anomaly; LRU is a stack algorithm but costs a list update per hit; Clock approximates it with a bit and a hand, PostgreSQL with a counter up to 5.
5. A use-once scan floods LRU and evicts the hot set; a 32-frame ring (sequential scans of tables larger than `shared_buffers`/4) confines it at no cost to the scan.
6. Steal + no-force is fastest and needs undo and redo in a write-ahead log, written before the data and forced at commit. `Buffers:` hit is the pool, read is OS cache *or* disk; measure with `pg_buffercache`, `pg_statio`, `pg_prewarm`.